Web Technologies

.....AND SECURITY

Stones Dalitso Chindipha

Chapter 6

JavaScript: Client-Side Scripting

Objectives

1. What is JavaScript
2. JavaScript Design
3. Using JavaScript
4. JavaScript Syntax
5. JavaScript Objects
6. The DOM
7. JavaScript Events
8. Forms

JavaScript Objects

- JavaScript is not a full-fledged object-oriented programming language.
- It does not have classes per se, and it does not support many of the patterns you'd expect from an object-oriented language like inheritance and polymorphism.
- The language does, however, support objects.
- In JavaScript, objects are central hub. If you understand objects, you understand JavaScript.
- All JavaScript values, except primitives, are objects.
  - Primitive values are immutable (they are hardcoded and cannot be changed).
  - A primitive value is a value that has no properties or methods.
    - 3.14 is a primitive value
    - A primitive data type is data that has a primitive value
  - JavaScript defines 7 types of primitive data types:
    - String, number, Boolean, null, undefined, symbol, bigint

JavaScript Objects …

- In JavaScript, almost "everything" is an object.
  - Booleans can be objects (if defined with the **new** keyword)
  - Numbers can be objects (if defined with the **new** keyword)
  - Strings can be objects (if defined with the **new** keyword)
  - **Dates** are always objects, **Maths** are always objects, **Regular expressions** are always objects, **Arrays** are always objects, **Functions** are always objects
  - **Objects** are always objects
- Objects can have **constructors, properties**, and **methods** associated with them.
- There are objects that are included in the JavaScript language.
  - You can also define your own kind of objects.

Using an Object Literal

- An object literal is a list of **name:value** pairs inside curly braces **{}**.
  - It is the easiest way to create a JavaScript Object.
- Using an object literal, you both define and create an object in one statement.
  - Spaces and line breaks are not important.
  - An object definition can span multiple lines:

```
const person = {firstName:"John", lastName:"Doe", age:50, eyeColor:"blue"};
```

```
const person = {
  firstName: "John",
  lastName: "Doe",
  age: 50,
  eyeColor: "blue"
};
```

```html
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Objects</h2>
<p>Creating a JavaScript Object:</p>

<p id="demo"></p>

<script>
const person = {firstName:"John", lastName:"Doe", age:50,eyeColor:"blue"};

document.getElementById("demo").innerHTML =
person.firstName + " is " + person.age + " years old.";
</script>

</body>
</html>
```

JavaScript Objects

Creating a JavaScript Object:

John is 50 years old.

JavaScript Object Properties

- Properties are the most important part of any JavaScript object.
- Properties are the values associated with a JavaScript object.
- A JavaScript object is a collection of unordered properties.
- Properties can usually be changed, added, and deleted, but some are read only.
- Each object might have properties that can be accessed, depending on its definition.
- When a property exists, it can be accessed using **dot notation** where a dot between the instance name and the property references that property.
  - **Dot notation** is not the only way but it's the most common one
  - The expression must evaluate to a property name.

Syntax For Accessing The Property Of An Object

```
objectName.property       // person.age
```

or

```
objectName["property"]    // person["age"]
```

or

```
objectName[expression]    // x = "age"; person[x]
```

```html
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Properties</h2>
<p>Access a property with .property:</p>

<p id="demo"></p>

<script>
const person = {
  firstname: "John",
  lastname: "Doe",
  age: 50,
  eyecolor: "blue"
};

document.getElementById("demo").innerHTML = person.firstname + " is " + person.age
+ " years old.";
</script>

</body>
</html>
```

```html
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Properties</h2>

<!-- Access a property with ["property"] -->

<p id="demo"></p>

<script>
const person = {
  firstname: "John",
  lastname: "Doe",
  age: 50,
  eyecolor: "blue"
};

document.getElementById("demo").innerHTML = person["firstname"] + " is " +
person["age"] + " years old.";
</script>

</body>
</html>
```

```html
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Objects</h2>
<p>Creating a JavaScript Object:</p>

<!-- This example creates an empty JavaScript object, and then adds 4 properties--
>

<p id="demo"></p>

<script>
const person = {};
person.firstName = "John";
person.lastName = "Doe";
person.age = 50;
person.eyeColor = "blue";

document.getElementById("demo").innerHTML =
person.firstName + " is " + person.age + " years old.";
</script>

</body>
</html>
```

More options for Properties?

- Adding New Properties
  - You can add new properties to an existing object by simply giving it a value.
- Deleting Properties
  - The **delete** keyword deletes both the value of the property and the property itself.
  - After deletion, the property cannot be used before it is added back again.
  - The delete operator is designed to be used on object properties.
    - It has no effect on variables or functions.
  - **The delete operator should not be used on predefined JavaScript object properties. It can crash your application.**

```html
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Properties</h2>
<!-- Add a new property to an existing object -->

<p id="demo"></p>

<script>
const person = {
  firstname: "John",
  lastname: "Doe",
  age: 50,
  eyecolor: "blue"
};

person.nationality = "English";
document.getElementById("demo").innerHTML =
person.firstname + " is " + person.nationality + ".";
</script>

</body>
</html>
```

```html
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Properties</h2>
<!-- Deleting object properties -->

<p id="demo"></p>

<script>
const person = {
  firstname: "John",
  lastname: "Doe",
  age: 50,
  eyecolor: "blue"
};

delete person.age;

document.getElementById("demo").innerHTML =
person.firstname + " is " + person.age + " years old.";
</script>

</body>
</html>
```

JavaScript Methods

- JavaScript methods are actions that can be performed on objects
  - method is a property containing a function definition.
  - Methods are **functions** associated with an instance of an object.
  - **functions** stored as object properties
- These methods are called using the same **dot notation** as for properties, but instead of accessing a variable, we are calling a method.
  - *`objectName.methodName()`*
    - someObject.**doSomething()**;
- Methods may produce different output depending on the object they are associated with because they can utilize the internal properties of the object.
- **If you access the fullName property, without (), it will return the function definition:**

Adding a Method to an Object

- Adding a new method to an object is straight forward:
  - ```
    person.name = function () {
      return this.firstName + " " + this.lastName;
    };
    ```
- **Using Built-In Methods:** many built in functions available
  - **toUpperCase()** method of the String object, to convert a text to uppercase:
    - ```
      let message = "Hello world!";
      let x = message.toUpperCase();
      ```

```html
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Objects</h2>
<p>Creating and using an object method.</p>
<!-- A method is actually a function definition stored as a property value.
This example accesses the fullName() method of a person object -->

<p id="demo"></p>

<script>
const person = {
  firstName: "John",
  lastName: "Doe",
  id: 5566,
  fullName: function() {
    return this.firstName + " " + this.lastName;
  }
};

document.getElementById("demo").innerHTML = person.fullName();
</script>
</body>
</html>
```

```html
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Objects</h2>
<p id="demo"></p>

<!-- example uses the toUpperCase() method of the String object -->
<script>
const person = {
  firstName: "John",
  lastName: "Doe",
  id: 5566,
};
person.name = function() {
  return (this.firstName + " " + this.lastName).toUpperCase();
};

document.getElementById("demo").innerHTML =
"My father is " + person.name();
</script>

</body>
</html>
```

JS Object Constructors: Constructor function

- Sometimes we need a **"blueprint"** for creating many objects of the same "type".
- The way to create an **"object type",** is to use an object **constructor function**.
  - a **constructor function** is used to create objects.
- **Object Literal is generally used to create a single object. The constructor function is useful if you want to create multiple objects**
- Objects of the same type are created by calling the **constructor function**
  - Each object created from the constructor function is unique.
  - To create an object from a constructor function, we use the **new** keyword.

https://www.programiz.com/javascript/constructor-function https://www.w3schools.com/js/js_object_constructors.asp

```html
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Constructors</h2>

<p id="demo"></p>

<script>
// Constructor function for Person objects
function Person(first, last, age, eye) {
  this.firstName = first;
  this.lastName = last;
  this.age = age;
  this.eyeColor = eye;
}

// Create a Person object
const myFather = new Person("John", "Doe", 50, "blue");

// Display age
document.getElementById("demo").innerHTML =
"My father is " + myFather.age + ".";
</script>

</body>
</html>
```

- In JS, when this keyword is used in a constructor function, this refers to the object when the object is created.
  - when an object accesses the properties, it can directly access the property as Person.name
- this is not a variable. It is a keyword.
  - You cannot change the value of this.

Creating multiple objects with CS

```html
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Constructors</h2>

<p id="demo"></p>

<script>

// Constructor function for Person objects
function Person(first, last, age, eye) {
  this.firstName = first;
  this.lastName = last;
  this.age = age;
  this.eyeColor = eye;
}

// Create two Person objects
const myFather = new Person("John", "Doe", 50, "blue");
const myMother = new Person("Sally", "Rally", 48, "green");

// Display age
document.getElementById("demo").innerHTML =
"My father is " + myFather.age + ". My mother is " + myMother.age + ".";

</script>

</body>
</html>
```

It is considered a good practice to capitalize the first letter of your constructor function.

```html
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Constructors</h2>

<p id="demo"></p>

<!-- To add a new property to a constructor, you must add it to the constructor function: -->
<script>

// Constructor function for Person objects
function Person(first, last, age, eye) {
  this.firstName = first;
  this.lastName = last;
  this.age = age;
  this.eyeColor = eye;
  this.nationality = "English";
}

// Create 2 Person objects
const myFather = new Person("John", "Doe", 50, "blue");
const myMother = new Person("Sally", "Rally", 48, "green");

// Display nationality
document.getElementById("demo").innerHTML =
"My father is " + myFather.nationality + ". My mother is " + myMother.nationality;
</script>

</body>
</html>
```

JavaScript Objects are Mutable

- Objects are mutable
- They are addressed by reference, not by value.
  - E.g. `const x = person;  // Will not create a copy of person.`
  - Both x and person are the same object.
  - Any changes to x will also change person

```
const person = {
  firstName:"John",
  lastName:"Doe",
  age:50, eyeColor:"blue"
}

const x = person;
x.age = 10;      // Will change both x.age and person.age
```

Adding a Method to a Constructor

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Constructors</h2>

<p id="demo"></p>

<script>

// Constructor function for Person objects
function Person(first, last, age, eye) {
  this.firstName = first;
  this.lastName = last;
  this.age = age;
  this.eyeColor = eye;
  this.name = function() {
    return this.firstName + " " + this.lastName
  };
}

// Create a Person object
const myFather = new Person("John", "Doe", 50, "blue");

// Display full name
document.getElementById("demo").innerHTML =
"My father is " + myFather.name();

</script>

</body>
```

Adding methods to an object constructor must be done inside the constructor function

```html
<body>
<h2>JavaScript Object Constructors</h2>

<p id="demo"></p>

<!-- In this example The changeName() function assigns the value of name to the
person's lastName property. -->

<script>
// Constructor function for Person objects
function Person(firstName,lastName,age,eyeColor) {
  this.firstName = firstName;
  this.lastName = lastName;
  this.age = age;
  this.eyeColor = eyeColor;
  this.changeName = function (name) {
    this.lastName = name;
  }
}
// Create a Person object
const myMother = new Person("Sally","Rally",48,"green");

// Change last name
myMother.changeName("Doe");

// Display last name
document.getElementById("demo").innerHTML =
"My mother's last name is " + myMother.lastName;
</script>
</body>
```

You cannot add a new method to an object constructor the same way you add a new method to an existing object.

Built-in JavaScript Constructors

- JavaScript has built-in constructors for native objects:
  -
```
new String()    // A new String object
new Number()    // A new Number object
new Boolean()   // A new Boolean object
new Object()    // A new Object object
new Array()     // A new Array object
new RegExp()    // A new RegExp object
new Function()  // A new Function object
new Date()      // A new Date object
```

Objects Included in JavaScript

- A number of useful objects are included with JavaScript including:
- **Array, Boolean, Date, Math, String, Window, Document Object Model (DOM) objects**
- **Reading assignment: Array, Boolean, Date, Math, String, Window (Pages 259 - 261)**